전체 소스 코드

> **Agent: Engineer** | 구현 단계

---

lib/types.ts

```typescript
export interface Receipt {
  id: string;
  date: string;
  storeName: string;
  supplyAmount: number;
  taxAmount: number;
  totalAmount: number;
  imageBase64?: string;
  status: 'pending' | 'synced' | 'error';
  createdAt: string;
  sheetsRowIndex?: number;
}

export interface AnalyzeResult {
  date: string;
  storeName: string;
  supplyAmount: number;
  taxAmount: number;
  totalAmount: number;
}

export interface ApiError {
  error: string;
  code: 'PARSE_FAILED' | 'API_ERROR' | 'SHEETS_ERROR' | 'VALIDATION_ERROR';
}
```

---

lib/validators.ts

```typescript
```

```typescript
import { z } from 'zod';

export const ReceiptSchema = z.object({
  date: z.string().regex(/^\d{4}-\d{2}-\d{2}$/, '날짜 형식: YYYY-MM-DD'),
  storeName: z.string().min(1, '상호명 필수'),
  supplyAmount: z.number().nonnegative(),
  taxAmount: z.number().nonnegative(),
  totalAmount: z.number().positive(),
});

export type ReceiptInput = z.infer<typeof ReceiptSchema>;

// 통화 문자열 → 숫자 변환
export const parseCurrency = (value: unknown): number => {
  if (typeof value === 'number') return value;
  if (typeof value === 'string') {
    const cleaned = value.replace(/[,원₩\s]/g, '');
    const parsed = parseFloat(cleaned);
    return isNaN(parsed) ? 0 : parsed;
  }
  return 0;
};
```

---

lib/openai.ts

```typescript
```

```
import OpenAI from 'openai';

export const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY,
});

export const RECEIPT_SYSTEM_PROMPT = `
당신은 영수증 이미지를 분석하는 전문 OCR 에이전트입니다.
이미지에서 다음 정보를 추출하여 반드시 아래 JSON 형식으로만 응답하세요.
마크다운 코드블록, 설명, 추가 텍스트 없이 순수 JSON만 출력하세요.

{
  "date": "YYYY-MM-DD",
  "storeName": "상호명",
  "supplyAmount": 숫자(공급가액, 부가세 제외),
  "taxAmount": 숫자(부가세),
  "totalAmount": 숫자(최종 합계)
}

규칙:
- 날짜가 없으면 오늘 날짜 사용
- 공급가액/부가세가 없으면 totalAmount의 10/11, 1/11로 계산
- 모든 금액은 정수(원 단위)
- 영수증이 아닌 이미지면: {"error": "영수증이 아닙니다"}
`;
```

---

lib/google-sheets.ts

```typescript
```

```
import { google } from 'googleapis';
import { Receipt } from './types';

const getAuthClient = () => {
  const keyJson = process.env.GOOGLE_SERVICE_ACCOUNT_KEY!;
  const credentials = JSON.parse(keyJson);

  return new google.auth.GoogleAuth({
    credentials,
    scopes: ['https://www.googleapis.com/auth/spreadsheets'],
  });
};

export const appendReceiptToSheet = async (receipt: Receipt): Promise<number> => {
  const auth = getAuthClient();
  const sheets = google.sheets({ version: 'v4', auth });

  const spreadsheetId = process.env.GOOGLE_SPREADSHEET_ID!;
  const sheetName = process.env.GOOGLE_SHEET_NAME ?? 'Sheet1';

  const values = [[
    receipt.id,
    receipt.date,
    receipt.storeName,
    receipt.supplyAmount,
    receipt.taxAmount,
    receipt.totalAmount,
    receipt.createdAt,
  ]];

  const response = await sheets.spreadsheets.values.append({
    spreadsheetId,
    range: `${sheetName}!A:G`,
    valueInputOption: 'USER_ENTERED',
    requestBody: { values },
  });

  const updatedRange = response.data.updates?.updatedRange ?? '';
  const match = updatedRange.match(/(\d+)$/);
  return match ? parseInt(match[1], 10) : -1;
};
```

app/api/analyze/route.ts

```typescript
```

```
import { NextRequest, NextResponse } from 'next/server';
import { openai, RECEIPT_SYSTEM_PROMPT } from '@/lib/openai';
import { ReceiptSchema } from '@/lib/validators';
import { parseCurrency } from '@/lib/validators';

export async function POST(req: NextRequest) {
  try {
    const { image } = await req.json();
    if (!image) {
      return NextResponse.json(
        { error: '이미지가 없습니다.', code: 'VALIDATION_ERROR' },
        { status: 400 }
      );
    }

    const response = await openai.chat.completions.create({
      model: 'gpt-4o',
      max_tokens: 500,
      messages: [
        { role: 'system', content: RECEIPT_SYSTEM_PROMPT },
        {
          role: 'user',
          content: [
            { type: 'image_url', image_url: { url: image, detail: 'high' } },
            { type: 'text', text: '이 영수증을 분석해주세요.' },
          ],
        },
      ],
    });

    const raw = response.choices[0]?.message?.content ?? '';

    let parsed: Record<string, unknown>;
    try {
      parsed = JSON.parse(raw);
    } catch {
      return NextResponse.json(
        { error: `JSON 파싱 실패. AI 응답: ${raw.slice(0, 100)}`, code: 'PARSE_FAILED'
        { status: 422 }
      );
    }

    if ('error' in parsed) {
      return NextResponse.json(
        { error: parsed.error as string, code: 'PARSE_FAILED' },
        { status: 422 }
```

```
      );
    }

    // 통화 변환
    const normalized = {
      date: parsed.date as string,
      storeName: parsed.storeName as string,
      supplyAmount: parseCurrency(parsed.supplyAmount),
      taxAmount: parseCurrency(parsed.taxAmount),
      totalAmount: parseCurrency(parsed.totalAmount),
    };

    const validated = ReceiptSchema.safeParse(normalized);
    if (!validated.success) {
      return NextResponse.json(
        { error: validated.error.message, code: 'VALIDATION_ERROR' },
        { status: 422 }
      );
    }

    return NextResponse.json(validated.data);
  } catch (err) {
    console.error('[analyze]', err);
    return NextResponse.json(
      { error: 'AI 분석 중 오류가 발생했습니다.', code: 'API_ERROR' },
      { status: 500 }
    );
  }
}
```

---

app/api/sheets/route.ts

```typescript
```

```typescript
import { NextRequest, NextResponse } from 'next/server';
import { appendReceiptToSheet } from '@/lib/google-sheets';
import { ReceiptSchema } from '@/lib/validators';

export async function POST(req: NextRequest) {
  try {
    const body = await req.json();

    const validated = ReceiptSchema.safeParse(body);
    if (!validated.success) {
      return NextResponse.json(
        { error: '유효하지 않은 데이터입니다.', code: 'VALIDATION_ERROR' },
        { status: 400 }
      );
    }

    const rowIndex = await appendReceiptToSheet(body);
    return NextResponse.json({ success: true, rowIndex });
  } catch (err) {
    console.error('[sheets]', err);
    return NextResponse.json(
      { error: 'Google Sheets 기록 중 오류가 발생했습니다.', code: 'SHEETS_ERROR' },
      { status: 500 }
    );
  }
}
```

---

hooks/useReceipts.ts

```typescript
```

```
'use client';
import { useState, useEffect } from 'react';
import { Receipt } from '@/lib/types';

const STORAGE_KEY = 'receipt_lens_data';

export const useReceipts = () => {
  const [receipts, setReceipts] = useState<Receipt[]>([]);
  const [isLoaded, setIsLoaded] = useState(false);

  // localStorage → state 복원
  useEffect(() => {
    try {
      const stored = localStorage.getItem(STORAGE_KEY);
      if (stored) setReceipts(JSON.parse(stored));
    } catch {
      console.warn('localStorage 복원 실패');
    }
    setIsLoaded(true);
  }, []);

  // state 변경 시 localStorage 동기화
  useEffect(() => {
    if (isLoaded) {
      localStorage.setItem(STORAGE_KEY, JSON.stringify(receipts));
    }
  }, [receipts, isLoaded]);

  const addReceipt = (receipt: Receipt) => {
    setReceipts(prev => [receipt, ...prev]);
  };

  const updateReceipt = (id: string, updates: Partial<Receipt>) => {
    setReceipts(prev =>
      prev.map(r => (r.id === id ? { ...r, ...updates } : r))
    );
  };

  const removeReceipt = (id: string) => {
    setReceipts(prev => prev.filter(r => r.id !== id));
  };

  const totalSynced = receipts
    .filter(r => r.status === 'synced')
    .reduce((sum, r) => sum + r.totalAmount, 0);
```

```
  return { receipts, addReceipt, updateReceipt, removeReceipt, totalSynced, isLoaded
};
```

---

components/ui/Button.tsx

```tsx
import { ReactNode } from 'react';
import { Loader2 } from 'lucide-react';

interface ButtonProps extends React.ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: 'primary' | 'secondary' | 'danger' | 'ghost';
  loading?: boolean;
  children: ReactNode;
}

const variants = {
  primary: 'bg-zinc-900 text-white hover:bg-zinc-700 disabled:bg-zinc-300',
  secondary: 'bg-white text-zinc-900 border border-zinc-200 hover:bg-zinc-50',
  danger: 'bg-red-500 text-white hover:bg-red-600',
  ghost: 'text-zinc-500 hover:text-zinc-900 hover:bg-zinc-100',
};

export const Button = ({ variant = 'primary', loading, children, className = '', dis
  <button
    className={`inline-flex items-center justify-center gap-2 px-4 py-2 rounded-lg t
    disabled={disabled || loading}
    {...props}
  >
    {loading && <Loader2 className="w-4 h-4 animate-spin" />}
    {children}
  </button>
);
```

---

components/ui/Badge.tsx

```tsx
```

```tsx
interface BadgeProps {
  status: 'pending' | 'synced' | 'error';
}

const config = {
  pending: { label: '대기중', className: 'bg-yellow-100 text-yellow-700' },
  synced: { label: '동기화됨', className: 'bg-green-100 text-green-700' },
  error: { label: '오류', className: 'bg-red-100 text-red-700' },
};

export const Badge = ({ status }: BadgeProps) => {
  const { label, className } = config[status];
  return (
    <span className={`px-2 py-0.5 rounded-full text-xs font-medium ${className}`}>
      {label}
    </span>
  );
};
```

---

components/scanner/ImageUploader.tsx

```tsx
```

```
'use client';
import { useRef } from 'react';
import { Camera, Upload } from 'lucide-react';
import { Button } from '@/components/ui/Button';

interface Props {
  onImage: (base64: string) => void;
}

export const ImageUploader = ({ onImage }: Props) => {
  const fileRef = useRef<HTMLInputElement>(null);
  const cameraRef = useRef<HTMLInputElement>(null);

  const handleFile = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onload = () => onImage(reader.result as string);
    reader.readAsDataURL(file);
  };

  return (
    <div className="flex flex-col items-center gap-4 p-8 border-2 border-dashed bord
      <div className="w-16 h-16 bg-zinc-100 rounded-full flex items-center justify-c
        <Camera className="w-8 h-8 text-zinc-400" />
      </div>
      <p className="text-sm text-zinc-500 text-center">
        영수증 사진을 업로드하거나 카메라로 촬영하세요
      </p>
      <div className="flex gap-2">
        <Button variant="secondary" onClick={() => fileRef.current?.click()}>
          <Upload className="w-4 h-4" /> 파일 선택
        </Button>
        <Button variant="secondary" onClick={() => cameraRef.current?.click()}>
          <Camera className="w-4 h-4" /> 카메라
        </Button>
      </div>
      <input ref={fileRef} type="file" accept="image/*" className="hidden" onChange=
      <input ref={cameraRef} type="file" accept="image/*" capture="environment" clas
    </div>
  );
};
```

components/scanner/ResultForm.tsx

```tsx
```

```
'use client';
import { useState } from 'react';
import { Send, AlertCircle } from 'lucide-react';
import { Receipt } from '@/lib/types';
import { Button } from '@/components/ui/Button';
import { useReceipts } from '@/hooks/useReceipts';

interface Props {
  receipt: Receipt;
  onSuccess: () => void;
}

export const ResultForm = ({ receipt, onSuccess }: Props) => {
  const { updateReceipt } = useReceipts();
  const [syncing, setSyncing] = useState(false);
  const [error, setError] = useState('');

  const fmt = (n: number) => n.toLocaleString('ko-KR') + '원';

  const handleSync = async () => {
    setSyncing(true);
    setError('');
    try {
      const res = await fetch('/api/sheets', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(receipt),
      });
      const data = await res.json();
      if (!res.ok) throw new Error(data.error);
      updateReceipt(receipt.id, { status: 'synced', sheetsRowIndex: data.rowIndex })
      onSuccess();
    } catch (e: unknown) {
      const msg = e instanceof Error ? e.message : '전송 실패';
      setError(msg);
      updateReceipt(receipt.id, { status: 'error' });
    } finally {
      setSyncing(false);
    }
  };

  return (
    <div className="bg-white border border-zinc-200 rounded-2xl p-6 space-y-4">
      <h3 className="font-semibold text-zinc-900">추출 결과</h3>
      <dl className="space-y-2 text-sm">
        {[
```

```
          { label: '날짜', value: receipt.date },
          { label: '상호명', value: receipt.storeName },
          { label: '공급가액', value: fmt(receipt.supplyAmount) },
          { label: '부가세', value: fmt(receipt.taxAmount) },
          { label: '합계', value: fmt(receipt.totalAmount), bold: true },
        ].map(({ label, value, bold }) => (
          <div key={label} className="flex justify-between">
            <dt className="text-zinc-500">{label}</dt>
            <dd className={bold ? 'font-semibold text-zinc-900' : 'text-zinc-700'}>{
          </div>
        ))}
      </dl>
      {error && (
        <div className="flex items-start gap-2 p-3 bg-red-50 rounded-lg text-red-600
          <AlertCircle className="w-4 h-4 mt-0.5 shrink-0" />
          <span>{error}</span>
        </div>
      )}
      <Button className="w-full" onClick={handleSync} loading={syncing}
        disabled={receipt.status === 'synced'}>
        <Send className="w-4 h-4" />
        {receipt.status === 'synced' ? '이미 전송됨' : 'Google Sheets에 전송'}
      </Button>
    </div>
  );
};
```

---

app/scanner/page.tsx

```tsx
```

```
'use client';
import { useState } from 'react';
import { v4 as uuidv4 } from 'uuid';
import { AlertCircle, ArrowLeft } from 'lucide-react';
import Link from 'next/link';
import { ImageUploader } from '@/components/scanner/ImageUploader';
import { ResultForm } from '@/components/scanner/ResultForm';
import { Button } from '@/components/ui/Button';
import { useReceipts } from '@/hooks/useReceipts';
import { Receipt } from '@/lib/types';

export default function ScannerPage() {
  const { addReceipt, updateReceipt } = useReceipts();
  const [image, setImage] = useState<string>('');
  const [analyzing, setAnalyzing] = useState(false);
  const [error, setError] = useState('');
  const [currentReceipt, setCurrentReceipt] = useState<Receipt | null>(null);

  const handleImage = async (base64: string) => {
    setImage(base64);
    setAnalyzing(true);
    setError('');
    setCurrentReceipt(null);

    // pending 상태로 먼저 저장
    const tempId = uuidv4();
    const tempReceipt: Receipt = {
      id: tempId,
      date: new Date().toISOString().split('T')[0],
      storeName: '분석 중...',
      supplyAmount: 0,
      taxAmount: 0,
      totalAmount: 0,
      imageBase64: base64,
      status: 'pending',
      createdAt: new Date().toISOString(),
    };
    addReceipt(tempReceipt);

    try {
      const res = await fetch('/api/analyze', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ image: base64 }),
      });
      const data = await res.json();
```

```
      if (!res.ok) throw new Error(data.error);

      const updated: Receipt = { ...tempReceipt, ...data };
      updateReceipt(tempId, data);
      setCurrentReceipt(updated);
    } catch (e: unknown) {
      const msg = e instanceof Error ? e.message : 'AI 분석 실패';
      setError(msg);
      updateReceipt(tempId, { status: 'error', storeName: '분석 실패' });
    } finally {
      setAnalyzing(false);
    }
  };

  return (
    <div className="min-h-screen bg-zinc-50 p-4 max-w-lg mx-auto space-y-6">
      <header className="flex items-center gap-3 pt-4">
        <Link href="/">
          <Button variant="ghost" className="p-2">
            <ArrowLeft className="w-5 h-5" />
          </Button>
        </Link>
        <h1 className="text-xl font-semibold text-zinc-900">영수증 스캔</h1>
      </header>

      <ImageUploader onImage={handleImage} />

      {image && (
        <div className="rounded-2xl overflow-hidden border border-zinc-200">
          <img src={image} alt="영수증 미리보기" className="w-full object-contain max-
        </div>
      )}

      {analyzing && (
        <div className="flex items-center justify-center gap-3 p-8 text-zinc-500">
          <div className="w-5 h-5 border-2 border-zinc-300 border-t-zinc-900 rounded
          <span className="text-sm">AI가 영수증을 분석하고 있습니다...</span>
        </div>
      )}

      {error && (
        <div className="flex items-start gap-2 p-4 bg-red-50 border border-red-100 r
          <AlertCircle className="w-4 h-4 mt-0.5 shrink-0" />
          <div>
            <p className="font-medium">분석 실패</p>
            <p className="text-red-500 mt-0.5">{error}</p>
          </div>
```

```
        </div>
      )}

      {currentReceipt && !analyzing && (
        <ResultForm receipt={currentReceipt} onSuccess={() => setCurrentReceipt(null
      )}
    </div>
  );
}
```

---

app/page.tsx (Dashboard)

tsx

```
'use client';
import Link from 'next/link';
import { ScanLine, TrendingUp, CheckCircle, Clock } from 'lucide-react';
import { useReceipts } from '@/hooks/useReceipts';
import { Badge } from '@/components/ui/Badge';
import { Button } from '@/components/ui/Button';

export default function Dashboard() {
  const { receipts, totalSynced } = useReceipts();
  const fmt = (n: number) => n.toLocaleString('ko-KR');
  const syncedCount = receipts.filter(r => r.status === 'synced').length;
  const pendingCount = receipts.filter(r => r.status === 'pending').length;

  return (
    <div className="min-h-screen bg-zinc-50 p-4 max-w-lg mx-auto space-y-6">
      <header className="pt-6 pb-2">
        <p className="text-sm text-zinc-400 mb-1">ReceiptLens</p>
        <h1 className="text-2xl font-bold text-zinc-900">영수증 대시보드</h1>
      </header>

      {/* 요약 카드 */}
      <div className="grid grid-cols-3 gap-3">
        <div className="bg-white rounded-2xl p-4 border border-zinc-100">
          <TrendingUp className="w-4 h-4 text-zinc-400 mb-2" />
          <p className="text-xs text-zinc-500">총 지출</p>
          <p className="text-lg font-bold text-zinc-900 mt-0.5">{fmt(totalSynced)}원
        </div>
        <div className="bg-white rounded-2xl p-4 border border-zinc-100">
          <CheckCircle className="w-4 h-4 text-green-400 mb-2" />
          <p className="text-xs text-zinc-500">동기화</p>
          <p className="text-lg font-bold text-zinc-900 mt-0.5">{syncedCount}건</p>
        </div>
        <div className="bg-white rounded-2xl p-4 border border-zinc-100">
          <Clock className="w-4 h-4 text-yellow-400 mb-2" />
          <p className="text-xs text-zinc-500">대기중</p>
          <p className="text-lg font-bold text-zinc-900 mt-0.5">{pendingCount}건</p>
        </div>
      </div>

      {/* 스캔 버튼 */}
      <Link href="/scanner">
        <Button className="w-full py-3 text-base">
          <ScanLine className="w-5 h-5" /> 새 영수증 스캔
        </Button>
      </Link>
```

```
      {/* 영수증 목록 */}
      <section>
        <h2 className="text-sm font-medium text-zinc-500 mb-3">최근 영수증</h2>
        {receipts.length === 0 ? (
          <div className="text-center py-12 text-zinc-400 text-sm">
            스캔한 영수증이 없습니다
          </div>
        ) : (
          <ul className="space-y-2">
            {receipts.map(r => (
              <li key={r.id} className="bg-white rounded-2xl px-4 py-3 border border
                <div>
                  <p className="font-medium text-zinc-900 text-sm">{r.storeName}</p>
                  <p className="text-xs text-zinc-400 mt-0.5">{r.date}</p>
                </div>
                <div className="text-right space-y-1">
                  <p className="font-semibold text-zinc-900 text-sm">{fmt(r.totalAmo
                  <Badge status={r.status} />
                </div>
              </li>
            ))}
          </ul>
        )}
      </section>
    </div>
  );
}
```

---

app/layout.tsx

```tsx
```

```
import type { Metadata } from 'next';
import { Inter } from 'next/font/google';
import './globals.css';

const inter = Inter({ subsets: ['latin'] });

export const metadata: Metadata = {
  title: 'ReceiptLens',
  description: '영수증 AI 스캐너',
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="ko">
      <body className={`${inter.className} bg-zinc-50`}>{children}</body>
    </html>
  );
}
```